

Rumi

Pre-development spec

FOUNDATIONS

Overview & scope

Key decisions

Legal & brand protection

Architecture

Repo & folder structure

DATA

Database schema

Security & RLS

Inventory & concurrency

Testing, staging & monitoring

PRODUCT

Sitemap

Page-by-page spec

Shared components

Admin panel

Candle care & safety

Gifting

Reviews (phase 2)

INTEGRATIONS

Payments (Razorpay)

Tax & GST

Refunds & cancellations

Order tracking security

WhatsApp button

Email notifications

Images & storage

GROWTH & TRUST

SEO requirements

Analytics & funnel events

Email capture & social proof

Mobile-first & performance

SHIP IT

Free-tier stack

Environment variables

Deployment steps

Backup & recovery

Launch content checklist

Real-world costs

Open questions

Rumi — Build Spec

Everything a developer needs to build the Rumi candle store: a full frontend storefront, a Supabase-backed data layer, a private admin panel, on-site payments, and a WhatsApp channel — using only free-tier tools. Pair this with the **Rumi Brand Bible** for every color, font, and tone-of-voice decision; this document does not repeat those values, it applies them.

v1.1 added the business-critical edge cases a v1.0 read flagged as missing: legal/brand protection, tax & GST, inventory concurrency, explicit variant validation, refunds & cancellations, tracking security, SEO, required email notifications, backups, candle care & safety, a launch content checklist, analytics events, admin audit logging, gifting fields, a phase-2 reviews, mobile-first/performance rules.

v1.2 was produced by putting v1.1 in front of a four-seat review panel (developer, marketing, designer, salesperson) and adjudicating every suggestion against the free-tier-only, no-constraints already set. It adds: payment/webhook idempotency, sequence-based order-number and GST-invoice generation, a concrete rate-limiting mechanism for order tracking, stock-release scheduling mechanism, a testing/staging/CI section, CORS and input-validation rules, optional free error tracking, email capture as retention infrastructure, gifting's review and gift-wrap fields promoted from "deferred" to v1 (cheap, and gifting is a stated differentiator), honest low-stock surfacing on the PDP, checkout trust signals and up-front shipping visibility, a WhatsApp concierge framing, occasion-page SEO content, and pre-launch marketing/legal-labeling checklist items. Nothing on the original deferred list (coupons, wishlists, advanced shipping, Cloudinary, advanced analytics, customer accounts, corporate-gifting portal, recommendations) moved — the panel's suggestions were checked against that list and rejected where they conflicted with it.

What this covers

A candle e-commerce site with three parts, built as three folders in one repository: a public **storefront** (browse, cart, checkout, pay), a private **admin panel** (login, upload, order management — access restricted to the owner only), and a **backend** that is entirely managed (Supabase), so there is no server to patch or pay for. A floating WhatsApp button sits on every storefront page for support and pre-sale questions, alongside full on-site checkout.

Assumption stated up front

This spec assumes India as the primary market (INR pricing, Razorpay as the payment gateway, IST timing) based on context available about the business. If Rumi is sold outside India, flag it before development starts — it changes the payment gateway, currency handling, and shipping-zone logic.

Key decisions this spec makes

These were the open forks in scope. Each is decided below so development can start without re-litigating them; each is also cheap to revisit later if it turns out wrong.

DECISION	CHOSEN	WHY
Site model	Full e-commerce (on-site cart, checkout, and payment) with a private admin panel	Confirmed as the desired scope. The WhatsApp button stays, but as a support/enquiry channel, not the only way to reach the owner.
Hosting cost	Zero recurring cost	Every tool below is on a free tier suitable for a small commercial site (see the caveat under Free-tier stack options). The one unavoidable real-world cost is a custom domain name (see Real-world costs).
Visual direction	Luxury & elegant, premium/gifting positioning	Set in the Brand Bible; this document's components and admin UI follow the same tokens so the admin panel doesn't feel like a bolted-on generic dashboard.
Backend model	Backend-as-a-service (Supabase), not a hand-rolled Node/Express server	Removes an entire category of hosting cost and maintenance (no server to keep patched or paying for) while giving a real Postgres database, authentication, file storage, and serverless functions.
Shipping model (v1)	Flat rate, with a free-shipping threshold	A pincode/weight-based courier engine is real complexity with no payoff until sales volume justifies it. <code>site_settings</code> holds the actual flat rate and threshold so the admin can change them without a deploy — see Real-world costs and the Page-by-page spec's Checkout entry.

Legal & brand protection

The Brand Bible deliberately builds the name and story around the 13th-century poet Rumi as inspiration, not affiliation. Before spending real money on packaging, printing, inventory, or paid marketing under this name, work through this checklist — it's cheap to fix now and expensive to fix after print runs and social handles established:

ITEM	WHY IT MATTERS
Trademark search — "Rumi" in the relevant class (candles/home fragrance, and gifting/retail)	Checks whether the name is actually free to use and register in this category before it's printed on thousands of boxes.
Domain availability	<code>.com</code> / <code>.in</code> for the intended storefront and admin subdomains (see \$Real-world costs).
Social handle availability	Instagram/other handles matching the brand name — reserve them even before launch to prevent squatting.
Wordmark registrability	A distinctive typographic treatment (Brand Bible §5) doesn't by itself guarantee the logo can be registered — a designer/trademark lawyer should confirm once the wordmark is finalized.
"Inspired by," not "affiliated with"	All copy referencing the poet Rumi must stay in the register of inspiration — never imply partnership, endorsement, or estate affiliation. This is a copy-review rule, not just a legal one (Brand Bible §1).
Trademark registration plan	A filing timeline, even if registration itself happens after a soft launch — filing early establishes priority.
Packaging/label legal requirements	India's Legal Metrology (Packaged Commodities) Rules require specific mandatory declarations on a packaged product — MRP, net quantity, and the complete manufacturer/packer name and postal address (not just a phone number), plus month and year of manufacture. Name this rule explicitly when briefing a packaging designer or printer — see \$Candle care & safety and Brand Bible §10 for the relevant label content, but confirm the exact legal wording with a professional before print.

Not a legal opinion

This checklist tells a developer/founder what to go check; it is not a trademark clearance or legal sign-off. Get a proper search and packaging-label review from a professional (a trademark agent, and — for food-adjacent labeling rules that sometimes extend to home/candle products — a compliance consultant) before committing to print runs or paid ads.

Architecture

Three moving pieces, no traditional backend server:

Storefront

Next.js (React) site. Reads products directly from Supabase (public, read-only). Writes — placing an order — go through one serverless Edge Function, never straight to the database, so prices can't be tampered with client-side.

Admin panel

A second, separate Next.js app, deployed on its own subdomain (e.g. `admin.rumicandles.com`). Locked behind Supabase Auth, allow-listed to your email only. Full read/write to products, orders, and settings.

Backend

Supabase: Postgres database, authentication, file storage for product photos, and Edge Functions for the two operations that we never trust the browser — creating an order total and verifying a payment.

Request flow — placing an order

```
Customer browses storefront (reads products directly from Supabase, public + read-only)
|
▼
Customer adds items to cart (kept in browser storage until checkout)
|
▼
Checkout: customer submits cart contents + shipping address
|
▼
Edge Function "create-order"
  → re-fetches each product's REAL price from the database (ignores any price the browser sent)
  → computes subtotal + shipping fee = total
  → creates a Razorpay order for that total
  → inserts an `orders` row with payment_status = 'pending'
  |
  ▼
Razorpay Checkout opens in the browser, customer pays
  |
  ▼
Edge Function "verify-payment" (called by the browser AND by Razorpay's webhook, belt-and-braces)
  → verifies Razorpay's signature server-side
  → on success: payment_status = 'paid', order_status = 'confirmed'
  |
  ▼
Order confirmation page + (optional) WhatsApp / email notification
```

The one rule that matters most in this whole document

Never let the browser tell the backend what an order costs. Every price used to charge a customer must be re-read from the `products` table inside the Edge Function. Never trust from the cart the browser posted. This is the single most common security mistake in DIY e-commerce builds.

Repository & folder structure

One Git repository, three top-level folders, matching how the pieces are described above and deployed independently:

```
rumi/
├── frontend/                # Public storefront – Next.js (App Router)
│   ├── app/
│   │   ├── page.tsx        # Home
│   │   ├── shop/
│   │   ├── product/[slug]/
│   │   ├── cart/
│   │   ├── checkout/
│   │   ├── order-confirmation/[orderNumber]/
│   │   ├── track-order/
│   │   ├── about/
│   │   ├── contact/
│   │   └── policies/
│   ├── components/         # Button, ProductCard, WhatsAppButton, Nav, Footer, CartDrawer...
│   ├── lib/
│   │   ├── supabaseClient.ts # anon key only – public, read-only calls
│   │   └── formatCurrency.ts
│   ├── styles/tokens.css    # generated from the Brand Bible's tokens.json – do not hand-edit values
│   └── public/
├── admin/                  # Private admin panel – separate Next.js app
│   ├── app/
│   │   ├── login/
│   │   ├── dashboard/
│   │   ├── products/       (list, /new, /[id]/edit)
│   │   ├── orders/         (list, /[id])
│   │   ├── categories/
│   │   └── settings/
│   ├── components/
│   └── lib/supabaseAdminClient.ts # still anon key + logged-in session; NEVER ships the service role key to any browser bundle
├── backend/                # Supabase project – the "backend", entirely config + SQL, no server to run
│   ├── migrations/
│   │   ├── 0001_init_schema.sql
│   │   ├── 0002_rls_policies.sql
│   │   └── 0003_seed_categories.sql
│   ├── functions/
│   │   ├── create-order/index.ts # Edge Function – validates variants/stock, recalculates totals+tax, opens Razorpay order
│   │   ├── verify-payment/index.ts # Edge Function – verifies Razorpay signature, marks order paid, generates GST invoice number
│   │   ├── razorpay-webhook/index.ts # Edge Function – Razorpay's server-to-server callback
│   │   ├── refund-order/index.ts # Edge Function – cancellation + refund flow ($Refunds & cancellations)
│   │   ├── track-order/index.ts # Edge Function – rate-limited order lookup ($Order tracking security)
│   │   └── release-stale-orders/index.ts # Edge Function – pg_cron-scheduled stock release ($Inventory & concurrency)
│   └── config.toml
└── docs/                  # This document, the Brand Bible export, and any decision notes
```

Deployment matches the folders: `frontend/` and `admin/` are each their own Cloudflare Pages project (two subdomains, two deploys), and `backend/`'s migrations and functions are pushed to Supabase with its CLI. Nothing about this requires a monorepo tool — plain folders and two build configs are enough scale.

Database schema

Postgres, hosted by Supabase. Written as the actual SQL a developer runs as the first migration — copy, don't just paraphrase. Expanded from the first draft spec to cover tax/GST, gifting, and refunds as real columns, not afterthoughts — see the callout below the table.

```

-- categories
create table categories (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  slug text unique not null,
  description text,
  sort_order int default 0,
  created_at timestamptz default now()
);

-- products
create table products (
  id uuid primary key default gen_random_uuid(),
  category_id uuid references categories(id),
  name text not null,
  slug text unique not null,
  sku text unique,                -- for products with no variants; variants carry their own sku
  description text,
  scent_notes text[],             -- e.g. {"Amber","Sandalwood","Smoke"}
  ingredients text,
  care_instructions text,         -- admin-editable; falls back to a default set – see $Candle care & safety
  hsn_code text,                 -- GST HSN classification – see $Tax & GST
  burn_time_hours numeric,
  weight_grams numeric,
  price_inr numeric not null,
  compare_at_price_inr numeric,   -- optional strike-through "was" price
  stock_quantity int not null default 0,
  low_stock_threshold int,        -- null = use the site-wide default in site_settings
  is_active boolean not null default false, -- the admin panel's "go live" toggle
  is_featured boolean not null default false,
  image_paths text[] not null default '{}', -- paths in Supabase Storage, ordered
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);

-- product_variants (optional – sizes like "Small / 120g", "Large / 250g")
create table product_variants (
  id uuid primary key default gen_random_uuid(),
  product_id uuid references products(id) on delete cascade,
  name text not null,
  price_inr numeric not null,
  stock_quantity int not null default 0,
  is_active boolean not null default true, -- lets admin retire one size without deleting it
  sku text unique
);

-- orders
create table orders (
  id uuid primary key default gen_random_uuid(),
  order_number text unique not null,    -- e.g. "RUMI-000123" – see $Order tracking security on why this is NOT used as a lookup
  customer_name text not null,
  customer_email text not null,
  customer_phone text not null,
  shipping_address jsonb not null,      -- {line1, line2, city, state, pincode}
  billing_address jsonb,               -- null = same as shipping_address; GST invoices need this distinct

  -- money – every figure is a snapshot at order time, never recomputed later from today's product/tax settings
  subtotal_inr numeric not null,
  discount_inr numeric not null default 0,
  taxable_amount_inr numeric not null,  -- subtotal - discount
  tax_rate_percent numeric not null default 0,
  tax_amount_inr numeric not null default 0,
  shipping_fee_inr numeric not null default 0,
  total_inr numeric not null,           -- taxable_amount + tax_amount + shipping_fee
  gst_invoice_number text unique,       -- set once, on payment success – see $Tax & GST
  buyer_gstin text,                   -- optional, for a business buyer's input-credit invoice

  -- payment
  payment_status text not null default 'pending'
  check (payment_status in ('pending','paid','failed','refunded')),
  payment_gateway_order_id text,       -- Razorpay order id
  payment_gateway_payment_id text,     -- Razorpay payment id, once paid

  -- refunds – see $Refunds & cancellations; only the verify-payment/refund Edge Function writes these

```

```

refund_status text not null default 'none'
  check (refund_status in ('none','initiated','partial','completed','failed')),
refund_amount_inr numeric,
razorpay_refund_id text,
refund_reason text,
refunded_at timestamptz,

-- fulfilment
order_status text not null default 'placed'
  check (order_status in ('placed','confirmed','packed','shipped','delivered','cancelled')),
courier_name text,
tracking_number text,

-- gifting – see $Gifting
is_gift boolean not null default false,
gift_message text,
recipient_name text,          -- shown only when is_gift; distinct from customer_name on the shipping label
gift_wrap_requested boolean not null default false,  -- free, always-available signature kraft wrap – see $Gifting

notes text,
created_at timestamptz default now(),
updated_at timestamptz default now()
);

-- order_items – a snapshot of what was bought, so later price/name/tax edits never change past orders
create table order_items (
  id uuid primary key default gen_random_uuid(),
  order_id uuid references orders(id) on delete cascade,
  product_id uuid references products(id),
  variant_id uuid references product_variants(id),
  product_name text not null,
  variant_name text,
  hsn_code text,              -- snapshot, for the GST invoice line
  unit_price_inr numeric not null,
  quantity int not null,
  line_total_inr numeric not null
);

-- site_settings – admin-editable, single-row-per-key config (WhatsApp number, shipping fee, banner text, GST defaults)
create table site_settings (
  key text primary key,
  value jsonb not null
);

-- admins – allow-list of who may sign in to the admin panel (usually exactly one row)
create table admins (
  id uuid primary key references auth.users(id),
  email text unique not null
);

-- admin_audit_logs – who changed what, and when – see $Admin panel
create table admin_audit_logs (
  id uuid primary key default gen_random_uuid(),
  admin_id uuid references admins(id),
  action text not null,       -- e.g. "product.price_updated", "order.status_changed"
  entity_type text not null,  -- "product" | "order" | "site_settings" | ...
  entity_id uuid,
  old_value jsonb,
  new_value jsonb,
  created_at timestamptz default now()
);

-- reviews – schema designed now, feature switched off until real orders exist – see $Reviews
create table reviews (
  id uuid primary key default gen_random_uuid(),
  product_id uuid references products(id) on delete cascade,
  order_id uuid references orders(id),      -- null = not a verified purchase
  customer_name text not null,
  rating int not null check (rating between 1 and 5),
  review_text text,
  image_path text,
  is_verified_purchase boolean not null default false,
  is_published boolean not null default false,  -- admin approval gate – see $Reviews
  created_at timestamptz default now()
);

```

```

-- subscribers – email capture, launch-scope (not "advanced analytics", just a table + a Resend template)
create table subscribers (
  id uuid primary key default gen_random_uuid(),
  email text unique not null,
  source text, -- "footer" | "checkout" | "post_purchase" – which capture point it came from
  consented boolean not null default true,
  created_at timestamptz default now()
);

-- rate_limit_attempts – backs the /track-order rate limiting rule in $Order tracking security
create table rate_limit_attempts (
  key text primary key, -- e.g. "track-order:" || ip, or "track-order:" || order_number
  attempt_count int not null default 1,
  window_start timestamptz not null default now()
);

-- sequences – never generate order_number or gst_invoice_number by reading max()+1 in application code;
-- that's the same race condition $Inventory & concurrency warns about, applied to numbers that legally can't collide
create sequence order_number_seq;
create sequence gst_invoice_seq;

```

Never recompute a historical order

Every money column on `orders` and `order_items` is a snapshot taken at the moment of purchase — tax rate, discount, and price included. If a product's price or site's tax rate changes next month, past invoices must read exactly as they did on the day they were issued. The `create-order` Edge Function writes these once and never recalculates them from current settings.

Still fine to add later rather than at launch: a `coupons` table (code, discount_type, discount_value, active, expires_at) — the `discount_inr` column above is there to receive it whenever it's built.

Security & Row Level Security (RLS)

Supabase's database is reachable directly from the browser, so every table needs an explicit rule for who can read or write it — this is what actually keeps it safe, not the frontend code. Enable RLS on every table, then:

TABLE	PUBLIC (ANON) CAN...	ADMIN CAN...
<code>categories</code> , <code>products</code> , <code>product_variants</code>	Read rows where <code>is_active = true</code> only	Full read/write
<code>orders</code> , <code>order_items</code>	No direct access at all — every write happens inside the <code>create-order</code> Edge Function, which uses the service role key server-side	Full read/write
<code>site_settings</code>	Read only the handful of keys marked public (WhatsApp number, shipping fee, banner text)	Full read/write
<code>admins</code>	No access	Read own row only
<code>admin_audit_logs</code>	No access	Read-only — written only by Edge Functions and admin panel server actions, never edited or deleted by anyone including admins
<code>reviews</code>	Read rows where <code>is_published = true</code> only; no direct write (submissions go through an Edge Function so <code>is_verified_purchase</code> can't be forged)	Full read/write (approve/reject)
<code>subscribers</code>	Insert only (email + source) — no read, no update, no delete	Full read/write
<code>rate_limit_attempts</code>	No access at all — read/written only inside the <code>track-order</code> Edge Function using the service role key	No access needed

"Admin can" above means: a Postgres policy that checks `exists (select 1 from admins where id = auth.uid())` . There is deliberately no separate role for launch — one owner account is the whole admin allow-list; add roles later only if a second person genuinely needs access.

Never expose the service role key

Supabase issues two keys: an *anon* key (safe to ship in the frontend and admin panel bundles — RLS still applies) and a *service role* key (bypasses RLS entirely). The service role key belongs only inside the Edge Functions in `backend/functions/` , set as a Supabase secret — never in frontend code, never in a public repo, never an environment variable prefixed `NEXT_PUBLIC_` .

Inventory & concurrency

The same rule that protects pricing — never trust the browser, always re-check server-side — applies just as hard to stock. Two customers can click "Buy" on a candle within the same second; the backend must guarantee only one of them succeeds.

Atomic decrement, not "read then write"

The wrong way: read `stock_quantity`, check it's > 0 in application code, then write `stock_quantity - 1`. Two simultaneous requests can both read the starting value before either writes, and both "succeed." The right way is one conditional SQL statement that reads and writes atomically, letting Postgres's own locking settle the race:

```
-- inside the create-order Edge Function, per line item, inside one database transaction:
update products
set stock_quantity = stock_quantity - :quantity
where id = :product_id
  and stock_quantity >= :quantity    -- the guard that makes this safe
returning stock_quantity;

-- if this UPDATE affects 0 rows, there wasn't enough stock --
-- roll back the whole transaction and return "out of stock" to the browser
-- BEFORE a Razorpay order is ever created.
```

The same statement shape applies to `product_variants.stock_quantity` when the line item has a variant — decrement the variant's row, not the parent product's.

What happens on failure or cancellation

SITUATION	STOCK BEHAVIOR
Stock check fails inside <code>create-order</code>	Whole transaction rolls back — nothing is decremented, no Razorpay order is created, customer sees "not enough stock" before paying.
Razorpay payment fails or the customer abandons checkout	Stock was already decremented when the order was created as <code>pending</code> . A scheduled job (or the next successful order's completion step) releases stock back for any order still <code>pending</code> after a short window (e.g. 30 minutes) with no successful payment.
Order cancelled by customer/admin after payment, before shipping	Stock is released back (<code>stock_quantity + quantity</code>) as part of the same transaction that sets <code>order_status = 'cancelled'</code> and starts the refund (\$Refunds).
Order cancelled after shipping	Stock is <i>not</i> auto-released — that's a return, handled manually once the item is actually back, not assumed.

The `stock_quantity >= :quantity` guard also makes negative inventory structurally impossible — there's no code path that can push it below zero, rather relying on every caller to remember to check first.

Low stock & out of stock

A product (or variant) is "out of stock" purely from `stock_quantity = 0` — never a separate flag to keep in sync. "Low stock" compares `stock_quantity` to `products.low_stock_threshold`, falling back to a site-wide default in `site_settings` (e.g. 3) when the per-product value is null — this feeds the admin dashboard's low-stock count (\$Admin panel). The same comparison is honest, useful information on the storefront too, not just the admin: when a product is below its threshold, the product page shows "Only N left in this batch" — true data already being tracked, not a manufactured scarcity timer (\$Page-by-page or \$Component library).

Releasing stock on a timed-out pending order

The 30-minute release window above needs an actual mechanism, not just a description — name one rather than leaving it to be decided at implementation. Enable `pg_cron` and `pg_net` (both included free in every Supabase project) and schedule a `release-stale-orders` Edge Function to run every 5 minutes. It finds `orders` rows still `pending` older than the window, releases their stock, and sets `order_status = 'cancelled'` with a reason of "payment timeout".

Testing, staging & monitoring

Nothing above is an architectural flaw, but a few "last mile" implementation habits are what actually stop a schema change or a pricing-logic edit from reaching production broken.

- **A second, free Supabase project as staging.** Early development and testing should never run against the same project real orders will later land in. The schema via the same `backend/migrations/` SQL into a second free project, and point local development at that one until launch.
- **A minimal automated test suite (Vitest, free/open-source), not full coverage.** Focused on the money-critical paths only: `create-order`'s re-price, atomic stock decrement, tax calculation, and the refund amount math. These are exactly the functions where a silent regression costs real money.
- **GitHub Actions (free for this repo size) running that suite on every push**, before Cloudflare Pages deploys — a schema or pricing-logic change that a test never reaches production.
- **Optional, still free: error tracking.** Supabase's own log retention is short. Wiring a free-tier error tracker (e.g. Sentry's free plan) into all five Edge Functions plus one admin email alert on any exception in `verify-payment` or `refund-order`, turns a silent failed payment into something the owner actually cares about the same day rather than from an angry customer message.

Sitemap

Storefront (public)	Admin panel (private)
/ Home	/login Email + password (Supabase Auth)
/shop All products, filterable	/dashboard Today's orders, revenue, low-stock alerts
/shop/[category] One category	/products List + search
/product/[slug] Product detail	/products/new Upload a new candle
/cart Cart drawer's full-page equivalent	/products/[id]/edit Edit, toggle live, delete
/checkout Address + payment	/categories Manage categories
/order-confirmation/[orderNumber] Thank-you page	/orders List, filter by status
/track-order Look up an order by number + phone	/orders/[id] Update status, add tracking number
/about Brand story	/settings WhatsApp number, shipping fee, banner, social
/contact WhatsApp, email, simple form	
/policies Shipping, returns, privacy, terms — required by Razorpay's KYC review	

Page-by-page spec

Home

Hero (full-bleed lit-candle photograph, `charcoal` gradient overlay for text legibility, the brand's hero line — see Brand Bible §3 — and one primary button in the top right). Below it: a featured-products grid (pulls `is_featured = true`), a short brand-story block linking to `/about`, a "how it's made" strip (three photos: pouring, trimming, wrapping), and the footer. No pop-ups, no countdown timers — see Brand Bible §11.

Shop / category

A responsive grid of `ProductCard` components (see §Components). Filters: category (chips), scent family, price range — kept simple (no faceted search infrastructure needed at this catalog size). Sort: featured, price low–high, price high–low, newest.

Product detail

Image gallery (swipeable on mobile) on one side, on the other: name (display font), price with a cost-per-burn-hour line beneath it (see Brand Bible's pricing-communication guidance — a concrete, on-voice answer to "why this much" without a superlative), scent notes as small tags, burn time and weight as a con row, an honest low-stock line when `stock_quantity` is at/below threshold ("Only N left in this batch" — §Inventory & concurrency), a delivery-time estimate ("Delivers in X–Y days," the same `site_settings` value the confirmation page uses), variant picker (if variants exist, showing only variants where `is_active = true`), quantity stepper, **Add to Cart** (primary, gold) and **Ask on WhatsApp** (secondary, outlined) side by side. Below the fold: full description, ingredients, instructions block (see §Candle care & safety — pulled from `products.care_instructions`, falling back to the default set if empty), and a "you may also like" block of 4 related products from the same category. Once real orders exist, published reviews (§Reviews) render below the description.

Cart

Line items with thumbnail, name, variant, quantity stepper, remove. Subtotal, the **actual** flat shipping fee and free-shipping threshold from `site_settings` as the real figure here, not deferred to an "estimated" note at checkout, since it's a known flat rate with no reason to hide it — and a single **Checkout** button in the top right. State: a friendly line in brand voice, not a bare "Your cart is empty."

Checkout

One page, four sections top to bottom: shipping address (name, phone, email, address, city, state, pincode), a "this is a gift" toggle (reveals recipient name, message field, the gift-wrap checkbox, and a separate billing-address field when checked — see §Gifting), order summary (subtotal, discount if any, tax, shipping — collapsed on mobile, expandable), and the payment button. Directly above the payment button: a trust strip — a lock icon with "Secure payment via Razorpay" accepted-payment-method marks, and an inline link to the returns/refund policy (not only in the footer) — the one moment a first-time buyer with no reviews is deciding whether to hand over card details. Shipping fee for v1 is a flat rate with a free-shipping threshold, both read from `site_settings` (not hard-coded; admin can tune them without a deploy). Submitting calls the `create-order` Edge Function first (see §Payments, §Tax & GST, §Inventory & concurrency) — Razorpay Checkout modal only opens once that function returns a valid order id.

Order confirmation

Order number, a checkmark moment (small, not a confetti animation — see Brand Bible §10 on motion), item summary with tax/total breakdown, delivery address, download/view link for the GST invoice (§Tax & GST), and both a "Continue shopping" link and a WhatsApp link pre-filled with the order number for post-purchase questions.

Track order

A small form (order number + phone number) that calls a dedicated Edge Function — never a public "list all orders" query, and never a client-side Supabase query against `orders` directly. See §Order tracking security for the exact rules (rate limiting, generic errors, no field beyond status/courier/tracking exposed).

Shared components

Built once, reused everywhere, styled from the Brand Bible's design tokens (never a hand-picked hex value in a component file). Button, Input, Badge, Product Nav, Footer, Toast, and the WhatsApp Button all have live, working previews in the Brand Bible's Components tab — build against those pixel-for-pixel rather than reinterpreting the spec below from scratch.

COMPONENT	SPEC
Button	Variants: primary (<code>gold</code> fill, <code>ink</code> text), secondary (outline, <code>wine</code> border and text), ghost (text only). <code>radius-full</code> . 250ms ease transition on hover. Brand Bible §10. Plus loading (flame-flicker label, used while <code>create-order</code> is in flight) and disabled (<code>disabled</code> token) states.
Input	Rest (<code>sand</code> fill), focus (<code>gold-deep</code> border + soft glow), error (<code>error</code> border, <code>paper</code> fill, specific message below — native browser validation suppressed in favor of this), and disabled (<code>disabled</code> token, locked during submission) — always labeled above the field, never placeholder-only.
Toast	Success/error variants, muted tint per Brand Bible §6, <code>radius-md</code> , no drop shadow. Cart-add confirmations and non-blocking errors.
Badge	New/seasonal (<code>wine</code>), natural-ingredient (<code>sage</code> tint), in-stock/sold-out (<code>success</code> / <code>error</code> tint, driven by <code>stock_quantity</code> , never hand-set), gift wrapped (<code>sand</code> + hairline border). One fixed meaning per color — never reused.
ProductCard	<code>sand</code> background, <code>radius-md</code> , <code>shadow-sm</code> resting / <code>shadow-md</code> hover. Image, name, price, a small "sold out" badge when <code>stock_quantity</code> is 0.
Nav	<code>paper</code> background, logo left, links center, cart icon with item-count badge right. Collapses to a hamburger + slide-out panel under 880px.
Footer	<code>charcoal</code> background, <code>charcoal-ink</code> text. Sitemap links, policies links, social icons, and a small "hand-poured in [city]" line.
WhatsAppButton	See §WhatsApp button below — its own section, since it has exact placement and behavior rules.
CartDrawer	Slides in from the right, <code>shadow-lg</code> , same line-item layout as the full cart page.

Admin panel

A separate app so the public storefront's bundle never ships a single line of admin code. Access: Supabase Auth email/password login, gated by the `admin` role (see §Security) — there is no public sign-up screen anywhere in this app. One account gates the entire business, so confirm two things are switched on (beyond default Supabase Auth behavior, not new work): its built-in password-reset-by-email flow, and exponential backoff/lockout after repeated failed logins.

Product upload flow (the core admin job)

- Admin fills in name, category, description, scent notes, ingredients, burn time, weight, price, stock quantity.
- Admin uploads 1–6 photos, which go straight to Supabase Storage's `product-images` bucket from the browser (using the logged-in admin session — the storage bucket restricts writes to admins).
- The product saves with `is_active = false` by default — a deliberate staging step.
- Admin reviews it on a "preview" link (the real product page, unlisted), then flips **Go live** — setting `is_active = true` — when ready. This is the one that actually makes a product visible to customers.

Order management

List view with filter chips for each `order_status` . Opening an order shows the full item list, customer/shipping/billing details, the gift message if `is_gift` , tax/invoice breakdown, and payment status (read-only — payment status is only ever set by the `verify-payment` function, never edited by hand, to avoid accidentally marking an unpaid order as paid). The admin can advance `order_status` (placed → confirmed → packed → shipped → delivered) and add a name + tracking number, or start a cancellation/refund (§Refunds & cancellations — this triggers the refund flow, it does not let the admin type a new `payment_status` directly).

Dashboard

Four numbers above the fold: today's orders, today's revenue, orders awaiting packing, and products at or below their low-stock threshold (§Inventory & counts). The site-wide default is configurable in Settings, per-product overrides on the product edit page). Kept to numbers a small operation actually checks each morning; resist the urge to build a full analytics suite here; §Analytics & funnel events covers the separate, lightweight event tracking.

Audit log

Every price change, stock adjustment, order-status change, and settings edit made from the admin panel writes one row to `admin_audit_logs` (admin, action, old value, new value, timestamp) — written by the admin panel's server-side actions, never by client-side code, so it can't be bypassed. Not shown as its own page at launch (one admin doesn't need to audit themselves), but the table exists from day one so it's simply there — filterable in a future `/admin/audit-log` page the moment a second person gets admin access.

Payments — Razorpay

Why Razorpay: among the mainstream Indian gateways, it charges no setup fee and no annual maintenance charge — only a per-transaction fee (about 2% standard domestic UPI/card payments as of late 2026; confirm the current rate on Razorpay's own pricing page before launch, since gateway pricing change). Cashfree's lower headline rate is offset by a real annual fee; Instamojo is a reasonable alternative with a similar zero-AMC structure. This transaction fee is unavoidable for any real payment gateway and is the one "cost" inside an otherwise free stack — it scales with sales, not a fixed bill.

Before writing any payment code

Razorpay requires business KYC before it will process live payments: PAN, bank account details, and basic business proof (even a sole-proprietorship declaration is enough to start). This is a paperwork step for the business owner, not a technical one — start it early, since verification can take a few days, and build against Razorpay test mode (fully free, no KYC needed) in the meantime.

Integration shape

1. **create-order** Edge Function, for each line item the browser sent:
 - Re-reads `product_id` (and `variant_id` if present) from the database — never trusts a name, price, or existence claim from the browser.
 - Confirms the variant actually belongs to that product, and that both product and variant have `is_active = true` — rejects the line item otherwise (a delisted product shouldn't be purchasable just because it's still in someone's cart).
 - Re-prices from `product_variants.price_inr` when a variant is present, else `products.price_inr` — the browser's price is never used for including as a sanity check.
 - Validates `customer_phone` (10-digit Indian mobile) and `shipping_address.pincode` (6 digits) with a strict regex before any write — malformed here breaks the courier handoff and SMS/email notifications later, so reject it at the boundary, not downstream.
 - Runs the atomic stock decrement from `$Inventory` & concurrency for that product/variant; aborts the whole function (no partial order) if any line item fails.
 - Generates `order_number` from `order_number_seq` (e.g. `'RUMI-' || lpad(nextval('order_number_seq')::text, 6, '0')`) inside the transaction — never `select max(...)+1`, which races exactly like ungated stock does.
 - Computes `taxable_amount = subtotal - discount`, then tax and total per `$Tax & GST`.
 - Creates a Razorpay order via their server-side API for the final `total_inr`, inserts the local `orders` + `order_items` rows as `pending`, and returns the Razorpay order id to the browser.
2. Browser opens Razorpay's Checkout widget with that order id — this is the only part of the integration that runs client-side, and it never sees a raw price or count it could tamper with.
3. **verify-payment** Edge Function: receives Razorpay's payment id + signature after checkout, verifies the signature server-side using the Razorpay secret and only then marks the order `paid` and generates the `gst_invoice_number`.
4. **razorpay-webhook** Edge Function: Razorpay also calls this directly (configured in the Razorpay dashboard) as a second, more reliable confirmation — the case where a customer closes the browser tab right after paying, before step 3 can run client-side.

The variant rule, stated plainly

Product price, variant price, and stock are always re-read server-side, immediately before creating the Razorpay order — never trusted from the cart the browser provided and never trusted from a variant id alone without confirming it belongs to the claimed product.

verify-payment must be idempotent

It's deliberately called twice — by the browser and by Razorpay's webhook — plus Razorpay itself retries webhook deliveries that don't return a 2xx. Without a guard, it can re-send confirmation emails, double-write the audit log, or race two writers on the same row. First line of the function: if `payment_status` is already `'paid'` for this order, return success immediately without re-running any side effect. Before acting on a webhook call, also check that `payment_gateway_payment_id` hasn't already been recorded, so a re-delivered webhook is a no-op.

CORS: `create-order` and `track-order` are the two Edge Functions callable by anyone, not just a logged-in admin — lock their `Access-Control-Allow-Origin` to the store's domain, not `*`, so a third-party site can't quietly call them to scrape live pricing/stock or hammer order lookups.

Tax & GST

Modeled as real fields from the start rather than bolted on later, because tax figures live inside historical orders forever (§Database schema's snapshot rule) Rumi is actually GST-registered at launch is a business decision (see §Open questions) — the schema supports both answers without a migration either way

FIELD (ON <code>ORDERS</code>)	MEANING
<code>subtotal_inr</code>	Sum of line items at their server-verified prices, before discount or tax.
<code>discount_inr</code>	Zero at launch (no coupon engine yet — §Database schema notes where one plugs in later).
<code>taxable_amount_inr</code>	<code>subtotal_inr - discount_inr</code> — the base GST is actually calculated on.
<code>tax_rate_percent</code> , <code>tax_amount_inr</code>	Read from <code>site_settings</code> at the moment of purchase (e.g. a single default GST rate for candles) and snapshotted onto the order; changing the site-wide rate next month never touches past orders.
<code>total_inr</code>	<code>taxable_amount_inr + tax_amount_inr + shipping_fee_inr</code> — the amount actually charged via Razorpay.
<code>gst_invoice_number</code>	Generated once, on payment success, from <code>gst_invoice_seq</code> (e.g. <code>'RUMI-INV-' nextval('gst_invoice_seq')</code>) inside <code>payment</code> 's transaction — a Postgres sequence, never a <code>max(...)+1</code> read in application code, since GST invoice numbers legally collide or be reassigned. Never reused, even if the order is later cancelled/refunded.
<code>buyer_gstin</code>	Optional free-text field for a business buyer who needs an input-tax-credit invoice; leave blank for a normal retail customer.
<code>billing_address</code>	Distinct from <code>shipping_address</code> — required for a correct invoice when gifting means the two addresses differ (§Gifting).
<code>products.hsn_code</code> / <code>order_items.hsn_code</code>	HSN classification for candles, set per product, snapshotted per order line for the invoice.

If GST registration is not yet in place at launch, set the site-wide `tax_rate_percent` to 0 — every column above still exists and still gets written as zero/blank; switching on GST later is a settings change, not a schema change.

Refunds & cancellations

Payment states (`pending` → `paid` → `failed` → `refunded`) and order states (`placed` → `confirmed` → `packed` → `shipped` → `delivered` → `cancelled`) are *what* happened; this section is *how* a cancellation actually moves money.

WHEN	WHAT HAPPENS
Customer abandons checkout before paying	Order stays <code>pending</code> ; no refund needed (nothing was charged). Stock is released after the timeout window in §Inventory & concurrency
Customer/admin cancels after payment, before shipping	Admin action triggers a refund-order Edge Function: calls Razorpay's refund API for the full amount, sets <code>refund_status = 'initiated'</code> , releases stock, sets <code>order_status = 'cancelled'</code> . Razorpay's webhook later confirms it, flipping <code>refund_status</code> to <code>'completed'</code> or <code>'failed'</code> , surfaced to the admin to retry manually).
Cancellation after shipping	Treated as a return, not an instant refund — out of this spec's automatic flow; the admin records the outcome once the courier/return person resolves, and issues a partial or full refund manually through the same Edge Function once the item is confirmed back.
Partial refund (e.g. one damaged item in a multi-item order)	Same Edge Function, called with an amount less than the order total; <code>refund_status = 'partial'</code> , <code>refund_amount_inr</code> records how much.

Fields written by this flow, all on `orders` : `refund_status` , `refund_amount_inr` , `razorpay_refund_id` , `refund_reason` , `refunded_at` . As with `payment_status` , the admin panel never lets anyone type these in directly — only the refund Edge Function and Razorpay's webhook write them, for the same reason a hand-editable "paid" flag would be dangerous.

Order tracking security

`/track-order` is the one place an unauthenticated stranger can look up someone else's order, so it needs its own rules — an order number that *looks* secret and guessable (`RUMI-000123`) must never be treated as a secret by itself.

- **Require two factors, not one.** Order number alone is not enough to return anything — always require it paired with the phone number (or email) on the request.
- **Generic errors.** "We couldn't find a matching order" for both "wrong order number" and "right order number, wrong phone" — never reveal which half is wrong, and never reveal that an order number exists at all before both factors match.
- **Rate limit by IP and by order number — a named mechanism, not a placeholder.** Inside the `track-order` Edge Function, atomically upsert a record `rate_limit_attempts` keyed on `ip` and separately on `order_number` ; past 5 attempts within a 10-minute `window_start` , reject with the same error below rather than a "too many attempts" message (which itself would confirm the order number is real). Reset the window once it expires.
- **Return the minimum.** Order status, courier name, and tracking number only — never the full address, payment details, or internal `orders.id` (expected to be a human-readable `order_number` everywhere public-facing; never the database uuid).
- **No listing endpoint, ever.** There is no query path, public or admin-adjacent, that returns "all orders for a phone number" without the matching order number on the request — that would turn a support feature into a way to enumerate someone's order history.

Gifting

The Brand Bible positions Rumi around gifting occasions (weddings, festivals, corporate hampers), so the database supports it from day one even though the UI for it can stay simple at launch:

- `orders.is_gift` and `orders.gift_message` — a checkbox + optional text field on Checkout (\$Page-by-page spec).
- `orders.recipient_name` , shown only when the gift toggle is on — without it, there's no way to know whose name goes on the shipping label versus the buyer's, and gifting stays a database field nobody actually sees.
- `orders.gift_wrap_requested` — one free checkbox: "Gift-wrap in signature kraft box" (Brand Bible \$10's packaging is already true for every order; no new packaging work required).
- `orders.billing_address` distinct from `shipping_address` — a gift order ships to the recipient but bills/invoices the buyer.
- Corporate/bulk order handling, gift bundles, and festival collection pages are real features worth having, and are deliberately deferred past v1 (see \$Open questions) so launch isn't blocked on them — recipient name and gift wrap moved out of that deferred list into v1 itself, since both are a column and a feature, not new architecture, and gifting is a stated brand differentiator.

Reviews — designed now, launched later

The `reviews` table (\$Database schema) exists from the first migration, but the feature stays switched off (no submission form, no display on product pages; no real order history to attach reviews to — launching with zero or fabricated reviews reads as worse than having none).

- **Verified purchase:** `is_verified_purchase` is only ever set `true` when the submitting customer's `order_id` actually contains that product and the order is complete — set server-side by a Submit-review Edge Function, never a client-settable flag.
- **Moderation:** every review starts `is_published = false` ; an admin approves it before it appears anywhere public.
- **Fields:** 1–5 star rating, review text, one optional photo (stored the same way as product images, \$Images & storage), and the review date.
- **Turning it on:** once there's a first batch of real, delivered orders, add a "leave a review" link to the post-purchase WhatsApp/email message (\$Email notifications) pointing at a simple form — no schema change needed at that point, only UI.

WhatsApp floating button

Zero-cost, no API key, no approval process: a plain `wa.me` link. Present on every storefront page (not the admin panel), fixed to the bottom-right corner.

PROPERTY	SPEC
Link format	<code>https://wa.me/<number-with-country-code, no plus or spaces>?text=<url-encoded pre-filled message></code>
Pre-filled message	Concierge framing, not a support fallback — this is a free, high-conversion pre-sale channel for exactly the moments a stranger hesitates on a gift (scent occasion, sizing). Product page: <i>"Hi Rumi, I'd like help picking between the Amber Dusk and something similar — it's a gift."</i> General contact page: <i>"Hi Rumi, I'm looking for a gift under ₹X — any recommendations?"</i> Homepage, near the button: one line of copy — "Not sure what to pick? Message us — real answers from our human bot."
Position	Fixed, bottom: 24px, right: 24px (16px on mobile, respecting the safe-area inset)
Style	<code>radius-full</code> , <code>shadow-lg</code> . Resting state uses the brand's <code>gold</code> ring per Brand Bible §10; the WhatsApp glyph itself keeps WhatsApp's own green, per brand guidelines — don't recolor the logo.
Number source	Read from <code>site_settings</code> (key <code>whatsapp_number</code>) so the admin can update it from <code>/admin/settings</code> without a code deploy.

Email notifications

Upgraded from "optional" in the first draft of this spec to a real launch requirement — WhatsApp is a fine conversational channel, but a real store needs a transaction record that doesn't depend on someone checking a chat thread. Sent via Resend's free tier (§Free-tier stack), triggered from the same Edge Functions that handle state changes — never a separate cron job re-deriving "what happened."

To the customer	To the admin
Order confirmation (on <code>create-order</code> success) · Payment confirmation + GST invoice link (on <code>verify-payment</code> success) · Order packed · Order shipped, with tracking number · Order delivered, with one added line item inviting an unboxing photo tagged to the brand handle (cheap, organic social proof while reviews are still off — §Email capture & social proof) · Refund confirmation (on refund completion).	New order notification · Payment failure notification · Low-stock alert (reuses the §Inventory & concurrency threshold — sent once when a product crosses it, not on every subsequent sale).

Every one of these is a template plus a trigger point that already exists in the Edge Functions above — this is a launch-scope requirement, not a new architecture.

Images & storage

Product photos live in a Supabase Storage bucket (`product-images`, public-read / admin-write), which fits comfortably in the 1 GB free-tier storage allowance for a candle catalog's photography. Serve images through Next.js's built-in image component so they're resized and lazy-loaded automatically; this alone removes the need for a separate image CDN. If the catalog grows past a few hundred products with many photos each, add Cloudinary's free tier (25 monthly credits, 1,000 transformations, 1 GB storage, or 1 GB bandwidth, in any mix) purely for on-the-fly transformations; it's an upgrade path, not a launch requirement.

Candle care & safety

The Brand Bible mentions wick-trimming as a voice example (§10); this section expands it into the actual content every product needs, and makes it admin- rather than hard-coded into page templates.

CONTENT	SOURCE
First-burn instructions (burn long enough for an even "melt pool" the first time)	Default set, overridable per product via <code>products.care_instructions</code>
Wick trimming (trim to 5mm before every light)	Default set
Maximum burn duration per session	Default set, or product-specific if a scent/wax needs a difference
Never leave a burning candle unattended; keep away from children and pets; keep away from flammable objects	Default set — required safety copy, same on every product
Container handling (jars get hot; discontinue use with roughly 1cm of wax left)	Default set
Storage instructions (cool, out of direct sunlight)	Default set

Implementation: a single default care-instructions block lives in `site_settings` (key `default_care_instructions`), rendered on every product page; `products.care_instructions` overrides it only when a specific product needs different wording. Both are plain admin-editable text — never hard-coded in a product-page template, since safety copy is exactly the kind of content that gets revised.

SEO requirements

Missing from the first draft of this spec entirely — worth calling out explicitly since a visually-driven store lives or dies on whether its product photography actually discovered.

Every product page	Site-wide
SEO title & meta description (distinct from the on-page name/description — written for a search snippet, not a product card) · canonical URL · Open Graph image · product structured data (<code>schema.org/Product</code> : price, availability from <code>stock_quantity</code> , brand, SKU).	<code>sitemap.xml</code> (generated from <code>is_active</code> products/categories) · <code>robots.txt</code> · canonical URLs on every page · favicon (the flame of the Brand Bible §5) · Open Graph/Twitter card metadata · a real, on-brand page — not a framework default.

Product images: meaningful `alt` text per photo (not the filename), served as WebP/AVIF with explicit width/height attributes so the layout doesn't shift when load (Next.js's built-in image component handles the format conversion and sizing automatically — §Images & storage).

Occasion content, not just technical SEO: schema markup and a sitemap don't rank a brand-new domain by themselves — the Brand Bible's own gifting (weddings, Diwali, housewarmings) are winnable long-tail searches ("candle gift for housewarming India") this catalog can target from day one. No new architecture add 2–3 occasion rows to `categories` (Diwali, Wedding Favors, Housewarming) with SEO-written copy, linked from Home and the Shop nav — reusing `/shop/[category]`, not a new route.

Analytics & funnel events

The admin dashboard deliberately stays to four numbers (§Admin panel) — that's a decision about the *dashboard*, not about whether to track anything at all. For event-level tracking, the business knows total sales but not where customers actually drop off.

Homepage viewed
Product viewed
Add to cart
Checkout started
Payment initiated
Payment successful (= Purchase)
WhatsApp button clicked

That short list is enough to answer the question that actually matters early on — *where in the funnel are people leaving* — e.g. "1,000 visitors → 150 products → 50 carts → 20 checkouts → 12 purchases" tells you to fix the product-page-to-cart step, long before a full analytics suite would. A free tier of any mainstream analytics tool (or even a handful of custom events into a Supabase table) is enough at this scale — the point is having the seven events at all, not which tool renders them.

Email capture & social proof

With no customer accounts and no coupons/loyalty program at launch (all correctly deferred — \$Open questions), email is the only durable audience asset the business builds before v2. Without capturing it, every visitor who doesn't buy today is gone for good. This is retention infrastructure, not an analytics platform (`subscribers` , \$Database schema) and one Resend welcome template.

Capture points A footer signup strip on every page ("Get first access to new scents — no spam, just light") · an opt-in checkbox on Checkout, checked by default but clearly optional · a post-purchase opt-in on the order confirmation page. Each writes one row to <code>subscribers</code> with its <code>source</code>.	Social proof while reviews are dark Reviews are correctly phase-2 (\$Reviews), which leaves a premium unfamiliar brand with no trust signal at exactly the moment it needs it. Cheap substitutes, all launch-scope: a founder photo + one-line story <code>/about</code>, the batch/pour-date already in the data model shown visit the PDP (not just in photography), and an embedded Instagram feed homepage.
---	---

Mobile-first & performance

A gifting-led, Instagram-discovered candle brand should expect the majority of its traffic on a phone, not a desktop — design and build for that first, not as an afterthought.

- **Design mobile-first:** lay out the phone view first, then expand to tablet/desktop — not the reverse.
- **Minimum tap target:** 44×44px on every interactive element (Brand Bible's Button component already meets this).
- **Product gallery:** swipeable, not a hover-to-zoom pattern that doesn't exist on touch.
- **Sticky Add to Cart:** on the product page, the price + Add to Cart button stays reachable while scrolling through a long description on mobile.
- **Checkout usability:** correct input types (`tel` for phone, `numeric` for pincode) so mobile keyboards show the right layout; autofill-friendly field names.
- **WhatsApp button safe area:** 16px inset plus `env(safe-area-inset-bottom)` so it never sits under a phone's home-bar gesture area (Brand Bible WhatsApp component spec).
- **Image ratios:** fixed aspect ratios reserved before the image loads, so the page doesn't jump (ties directly into the SEO section's layout-shift point).
- **Mobile typography:** body text no smaller than 15px on phone; the type scale in the Brand Bible already reflects this floor.
- **Performance budget:** a plain, reasonable target — a product page interactive in under ~3 seconds on a mid-range phone over 4G — kept realistic by and hosting choices already in this spec (Next.js image optimization, Cloudflare's CDN), not by adding a new tool.

Free-tier stack

Every service below has a free tier that explicitly allows commercial/business use — this was checked deliberately, because some popular hosts (notably Vercel's Hobby plan) restrict their free tier to personal, non-commercial projects. Limits below are current as of research for this document (September 2026) and are checked before launch, since providers do revise free-tier terms.

NEED	TOOL	FREE TIER	COMMERCIAL
Storefront hosting	Cloudflare Pages	Unlimited sites & bandwidth, 500 builds/month, 100,000 Workers requests/day	NO NON-COMMERCIAL CLAUSE
Admin panel hosting	Cloudflare Pages (second project)	Same as above	NO NON-COMMERCIAL CLAUSE
Database + Auth + Storage + Edge Functions	Supabase	500 MB database, 1 GB file storage, 50,000 monthly active users, 500,000 Edge Function calls/month; free projects pause after 7 days of inactivity (a scheduled ping avoids this)	ALLOWED
Payments	Razorpay	No setup fee, no annual fee — ~2% per transaction (unavoidable, scales with sales, see §Costs)	BUSINESS KYC REQUIRED
Customer contact	WhatsApp click-to-chat	Free, no account or API needed	ALLOWED
Image transforms (optional, later)	Cloudinary	25 credits/month (≈1,000 transforms, or 1 GB storage, or 1 GB bandwidth)	ALLOWED
Transactional email (§Email notifications)	Resend	3,000 emails/month free tier (verify current limit before relying on it) — comfortably covers a small store's order-confirmation and status-update volume	ALLOWED
Error tracking (optional, §Testing)	Sentry	Free developer tier — enough event volume for a store this size	ALLOWED
CI (optional, §Testing)	GitHub Actions	Free minutes/month, comfortably enough for a small test suite on every push	ALLOWED

Hosting note: Vercel is excellent for Next.js, but its own documentation states the free Hobby plan's fair-use terms restrict it to "non-commercial, personal use only" — a real risk for stores that take payments. Cloudflare's published terms carry no equivalent line, and running a business site on its free plan is common practice, but Cloudflare has not published an explicit "yes: yes" statement either — re-read Cloudflare's current Pages terms before launch as a final check, rather than treating this document's read as the last word.

Environment variables

VARIABLE	WHERE	NOTES
NEXT_PUBLIC_SUPABASE_URL	frontend, admin	Safe to expose
NEXT_PUBLIC_SUPABASE_ANON_KEY	frontend, admin	Safe to expose — RLS is what actually protects data
SUPABASE_SERVICE_ROLE_KEY	Supabase Edge Function secrets only	Never in frontend or admin code, never committed to git
RAZORPAY_KEY_ID	Edge Function secrets + frontend (publishable half)	The key id alone is safe client-side; the secret below is not
RAZORPAY_KEY_SECRET	Edge Function secrets only	Server-side signature verification only
RAZORPAY_WEBHOOK_SECRET	Edge Function secrets only	Verifies the webhook payload actually came from Razorpay
RESEND_API_KEY	Edge Function secrets only	§Email notifications — never exposed client-side
SENTRY_DSN	Edge Function secrets (optional)	§Testing, staging & monitoring — only if error tracking is wired

Deployment steps

1. Create a free Supabase project. Run the migrations in `backend/migrations/` via the Supabase CLI or SQL editor.
2. Deploy the three Edge Functions (`create-order` , `verify-payment` , `razorpay-webhook`) with the Supabase CLI; set their secrets (service role keys, Razorpay keys) in the Supabase dashboard, not in code.
3. Create a free Razorpay account, start in test mode, and note the test key id/secret for local development.
4. Push `frontend/` to a Git repository; connect it to a new Cloudflare Pages project; set its environment variables (the `NEXT_PUBLIC_*` ones) in the Pages dashboard.
5. Repeat for `admin/` as a second Cloudflare Pages project, on a separate subdomain.
6. Point a custom domain's DNS at Cloudflare Pages (both the root domain to the storefront and an `admin.` subdomain to the admin panel) once a domain is purchased.
7. Complete Razorpay's business KYC, switch from test keys to live keys, and do one real end-to-end test order — including a test refund — before announcing launch.
8. Set up a free scheduled ping (e.g. a free cron service, or a Supabase-recommended GitHub Action) once a week against the Supabase project so it doesn't pause from inactivity — schedule the \$Backup & recovery data export on the same job.
9. Create a free Resend account and connect it in the Edge Functions for \$Email notifications.
10. Clone the schema into a second free Supabase project for staging (\$Testing, staging & monitoring); point local development at it, not at the production project.
11. Set up GitHub Actions to run the Vitest suite on push, and optionally connect a free Sentry project to the Edge Functions.
12. Start the \$Legal & brand protection checklist (trademark, domain, social handles) in parallel with the steps above, not after them — it's the slowest-moving item on this list.

Backup & recovery

Orders and customer data are business-critical the moment the first sale happens — this section exists so recovering from a mistake or an accidental deletion is a known procedure, not a scramble.

- **Database backups:** Supabase takes automatic daily backups even on the free tier, retained for a rolling window (verify the current retention period on Supabase's own plan page before relying on a specific number of days) — restore is a dashboard action, not a support ticket.
- **Storage backups:** product images in Supabase Storage aren't covered by the same point-in-time database restore; treat the admin's original photo files as the real backup (keep them on their own computer or a cloud drive, outside this stack) — Storage is a serving copy, not the archive.
- **If the Supabase project is deleted accidentally:** without a very recent manual export, this is not recoverable through the free tier — which is exactly why the next two points matter.
- **Export orders/customers regularly:** a simple scheduled export (weekly is enough at launch scale) of the `orders` , `order_items` , and `products` tables as a CSV or JSON file kept somewhere outside Supabase entirely — this is the cheap insurance policy against the "project deleted" case above.
- **Recovery procedure, written down before it's needed:** who has access to the Supabase account, where the latest export lives, and the exact steps to create the project from the `backend/migrations/` SQL plus the latest export — a paragraph in `docs/` , not tribal knowledge.

Launch content checklist

The architecture above is ready before the actual content is. Every product needs the following filled in before it's flipped live (\$Admin panel's "go live" toggle literal checklist an admin can work down per product:

FIELD	WHERE IT LIVES
Name, short description, full description	<code>products.name</code> , <code>description</code>
Scent notes, ingredients	<code>products.scent_notes</code> , <code>ingredients</code>
Burn time, weight	<code>products.burn_time_hours</code> , <code>weight_grams</code>
Price, compare-at price if applicable	<code>products.price_inr</code> , <code>compare_at_price_inr</code>
Stock, SKU, HSN code	<code>products.stock_quantity</code> , <code>sku</code> , <code>hsn_code</code>
4–6 photos, each with meaningful alt text	<code>products.image_paths</code> + the SEO section's alt-text requirement
Care instructions (or confirm the default set applies)	<code>products.care_instructions</code> — \$Candle care & safety
SEO title, SEO description	Per-product SEO fields — \$SEO requirements

Plus, once per site rather than per product: the `/policies` page's actual wording (shipping, returns, privacy, terms), the default care-instructions block, the shipping rate and free-shipping threshold, and the WhatsApp number — all in `site_settings`, all listed again in \$Open questions as the remaining business decisions.

Also once per site, zero-cost, and easy to forget precisely because nothing above blocks launch on them: claim a Google Business Profile, link the Instagram live site, and — if there's any gap between "site is ready" and "products are photographed" — put up a simple waitlist/coming-soon page rather than nothing

Real-world costs — the honest version

Everything above is genuinely free to run. Two things are not, and no configuration avoids them:

REAL COST A domain name A <code>.com</code> or <code>.in</code> domain typically runs somewhere around ₹500–1,200/year depending on registrar and extension. The zero-cost fallback is a free subdomain Cloudflare Pages issues automatically (like <code>rumi.pages.dev</code>) — usable for launch and testing, just less polished for a premium brand.	REAL COST Payment gateway transaction fee Razorpay's ~2% per transaction is not a subscription — it's a cut of sale, unavoidable for any gateway that actually moves real money. It's the closest thing to a "cost" in this stack, and it only exists when a sale happens.
--	---

Open questions for the business side

Not blockers for starting development — the architecture above supports every answer to these — but needed before launch, flagged here rather than guesses

- Final product catalog: exact scent list, sizes/variants, and pricing (this spec's schema supports variants; the actual list is a business decision).
- The actual flat shipping rate and free-shipping threshold (the *model* is decided — \$Key decisions — the numbers aren't).
- Return/refund policy wording, required for the `/policies` page and for Razorpay's KYC review (the refund *mechanics* are specified in \$Refunds & cancellations — the customer-facing policy text is a business/legal decision). Worth prioritizing over the other open questions here: for a fragile, premium time gift purchase with no reviews yet to reassure a buyer, this is the single biggest silent objection at checkout — see the trust-strip note in \$Page-by-product's Checkout entry.
- GST registration status and the actual tax rate to configure (the schema and checkout logic support either answer — \$Tax & GST).
- Business entity type for the Razorpay KYC paperwork (individual/sole proprietor vs. registered company).
- Trademark search, domain, and social-handle checks (\$Legal & brand protection) — start these in parallel with development, not after.
- The default care-instructions wording and the low-stock threshold default — both are simple `site_settings` values once decided.

Deliberately out of scope for this version, and fine to leave there: coupons/discount codes, a wishlist, a loyalty program, a pincode/weight-based shipping engine, Cloudinary, an advanced analytics dashboard, customer accounts/login, a dedicated corporate-gifting portal, and product recommendations. None of the architecture above blocks adding these later — several (discounts, reviews, gifting) already have a column or table waiting for them.

Pair with the **Rumi Brand Bible** for the full color palette, type scale, voice, imagery direction, and live component previews referenced throughout this document. Free-tier limits were confirmed via provider documentation and third-party trackers in September 2026 — reconfirm on each provider's own pricing page before committing, since these terms change over time. v1.0

September 2026.